

中期答辩逐页讲稿

目标时长：7 分钟以内

建议总时长：约 6 分 40 秒 - 7 分 00 秒

第 1 页：标题

建议时长：10 秒

讲稿：

各位老师好，我汇报的题目是《基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发》。本次中期答辩主要汇报目前已经完成的工作、实验案例，以及后续计划。

第 2 页：目录

建议时长：10 秒

讲稿：

接下来我将从五个部分展开，分别是预定计划的执行情况、已完成工作、实验案例、后期计划，以及目前存在的困难和按时完成的可能性。

第 3 页：预定计划的执行情况

建议时长：35 秒

讲稿：

项目整体是按照三个阶段推进的。第一阶段主要完成系统骨架的搭建，把任务状态、角色分工和运行对象先定义清楚。第二阶段重点是把系统主流程打通，使系统能够完成方案生成、工具调用、人工确认以及失败恢复。第三阶段主要是整理中期实验、过程证据和论文材料。

从当前进度看，系统主流程、人工确认流程和恢复流程都已经完成，中期机制验证基准也已经整理完成。尚未完成的工作主要是横向对比实验和外部模型训练。

第 4 页：已完成工作：系统骨架与角色分工

建议时长：35 秒

图示：

- `assets/split/multi-agent-core-part-1-core-orchestration.svg`

讲稿：

这一页展示的是系统骨架与角色分工。图中可以看到，系统以 `WorkflowRuntime` 作为统一运行核心，把 `Planner`、`Executor`、`Safety` 和 `Summarizer` 四类 Agent 组织起来。

其中 `Planner` 负责生成方案，`Executor` 负责执行步骤，`Safety` 负责风险和门禁判断，`Summarizer` 负责整理最终结果。这个结构说明系统已经不是若干零散功能的拼接，而是形成了统一的任务运行框架。

图示关键阐述：

- `WorkflowRuntime` 在图中央，说明它承担统一调度作用。
- 四类 Agent 分工明确，体现了系统不是单模型一步走到底。
- 这张图主要用来说明“架构已经搭好”。

第 5 页：已完成工作：关键运行机制

建议时长：40 秒

图示：

- `assets/split/multi-agent-core-part-2-planning-execution.svg`

讲稿：

这一页进一步说明系统已经打通了关键运行机制。当前已经实现从方案生成、工具选择到执行结果返回的主链，同时支持关键节点暂停、等待人工决策，以及在失败之后按照 `retry`、`patch`、`replan` 的顺序继续处理。

因此，系统现在不只是能规划，还能够真正执行、出错后恢复，并把过程中的关键事件记录下来，支持后续回放和分析。

图示关键阐述：

- 左侧是规划与执行主链，说明从计划到工具调用再到结果返回已经打通。
- 图中的工具相关对象说明系统支持多工具组织，而不是单一工具调用。
- 这张图重点讲“运行机制已经实现”，不是讲最终实验效果。

第 6 页：已完成工作：工程证据沉淀

建议时长：30 秒

图示：

- `assets/split/multi-agent-core-part-4-artifacts-output.svg`

讲稿：

除了功能实现本身，目前还已经形成了一套工程证据体系。图中展示了 `EventLog`、`TaskSnapshot` 和 `DesignResult` 三类关键对象，分别对应事件日志、过程快照和最终结果产物。

这意味着系统运行之后，不只是得到一个最终结果，而是能够保留过程证据，支持回放、追溯和解释。

图示关键阐述：

- `EventLog` 说明关键事件会被记录下来。
- `TaskSnapshot` 说明中间状态和计划版本可以保留。
- `DesignResult` 说明最终结果和报告文件能够沉淀下来。

第 7 页：实验案例一：人工确认与失败案例回放

建议时长：40 秒

图示：

- `assets/recovery-hitl-overview.pdf`

讲稿：

这一页是一个代表性的机制案例回放。这里想说明的不是单次结果有多好，而是系统在运行过程中如何进入等待态、接收人工确认，并在失败后继续恢复执行。

具体来说,在关键节点系统会显式进入 WAITING 状态;人工决策会进入 PendingAction、Decision 和 EventLog 这条证据链;遇到失败时系统会先做局部修补,必要时再升级为 replan;在恢复之后,任务还能够继续执行并收敛到终态。

图示关键阐述:

- 图中的等待态说明系统不是隐式连续执行。
- 决策回流链说明人工参与不是界面层面的临时操作,而是进入正式证据链。
- 恢复路径说明系统具备可恢复性,而不是一旦失败就整体终止。

第 8 页: 实验案例一: 人工确认与失败案例回放

建议时长: 35 秒

图示:

- asserts/hitl-decision.pdf

讲稿:

这一页展示的是人工决策页面。从界面上可以看到,系统会同时给出多个候选方案,并附带风险、成本、推荐项等信息,供人工进行选择。

这说明系统的人工参与不是简单的手工覆盖,而是在系统给出候选方案之后,由人工在关键节点完成有依据的确认。

图示关键阐述:

- 候选列表说明系统会给出多个可选方案,而不是单一路径。
- 风险、成本和推荐字段说明人工决策有结构化依据。
- 这一页重点讲“人工如何介入”,不是讲算法细节。

第 9 页: 实验案例一: 人工确认与失败案例回放

建议时长: 30 秒

图示:

- asserts/timeline-split.pdf

讲稿：

这一页展示事件时间线视图。它把任务运行过程中的关键事件按顺序串联起来，包括进入等待态、提交决策、应用决策和退出等待态等状态变化。

这张图主要用来证明系统的运行链条是可追踪、可回放的，而不是只能看到最终结果。

图示关键阐述：

- 时间线能看到状态变化顺序。
- 等待态和决策相关事件可以清楚对应起来。
- 这张图重点讲“可回放、可追溯”。

第 10 页：实验案例二：中期机制验证基准结果

建议时长：40 秒

图示：

- `assets/benchmark/family_summary.svg`

讲稿：

由于原有纵向实验结果在中期阶段还不足以支撑方法效果上的强结论，因此我补充整理了一组机制验证基准。这里的目标不是证明最终生物效果最优，而是验证系统最核心的控制与恢复闭环是否已经稳定成立。

这组基准共包含 17 个可复现场景，覆盖 5 个实验家族。从图中可以看到，Execution and Summary、Gate、HITL and Audit、Planner Routing、Recovery 这 5 个家族都全部通过。

图示关键阐述：

- Scenario Count 表示每个实验家族包含多少个可复现场景。
- Pass Rate 表示该家族机制检查通过的比例。
- Evidence Complete Rate 表示相关证据是否完整成立。

第 11 页：实验案例二：中期机制验证基准结果

建议时长：30 秒

图示：

- assets/benchmark/artifact_support.svg

讲稿：

这一页进一步展示证据文件的完整性。可以看到 event_log、snapshot 和 report 这三类关键产物的落盘完整率都是 100%。

因此，这组补充实验说明的不只是测试通过，还说明系统已经具备比较完整的证据支撑，这对中期答辩的展示和后续论文写作都很重要。

图示关键阐述：

- Expected Count 是理论应产出的证据数量。
- Present Count 是实际成功落盘的数量。
- Present Rate 说明证据链是否完整。

第 12 页：后期计划

建议时长：40 秒

讲稿：

后续工作主要分为四段。第一段是在三月底前补齐横向实验并统一评估口径。第二段是在四月上旬推进外部模型训练与系统收口。第三段是在四月下旬完善前端页面，使后台接口和前端展示对齐，并更直观地展示系统结果。第四段是在五月完成论文和答辩材料的定稿。

也就是说，后续工作虽然仍多，但主要是在现有系统和现有证据基础上继续补齐和整理，而不是重新推翻重做。

第 13 页：困难与按时完成的可能性

建议时长：35 秒

讲稿：

目前的主要困难在于横向对比实验还没有完成，统一评估口径和图表结果还需要继续收口，外部模型训练也尚未正式启动。同时，多工具真实运行还会受到远程平台和资源条件的影响，工程材料向论文表达的转写也还需要继续整理。

但从整体进度看，系统主体、关键控制机制和主要运行链路已经基本完成，剩余工作边界也比较清楚，因此我认为项目具备按时完成论文与参加答辩的现实基础。

第 14 页：结束页

建议时长：10 秒

讲稿：

我的汇报到这里结束，感谢各位老师聆听，请老师批评指正。

关于 Agent

Agent 是什么

Agent 是一个在授权范围内自主执行动作的实体，它围绕明确的目标感知环境、做出决策、并对环境或后续流程产生可观察的影响。Agent 的本质在于主动性、目标驱动和对系统状态的作用，而非实现方向或技术选型。

这里的主动性体现在：

每个 Agent 都能在明确的授权边界内，根据当前环境和目标自主地产生会影响后续流程的输出。

关于 Benchmark

Benchmark 是什么

Benchmark 是一种用于评估系统、算法和模型性能的标准化实验方法。它围绕固定任务、统一输入和明确指标，使得不同方法的表现可以量化、可比较，并对优化、改进或选择提供参考依据。Benchmark 的本质在于可重复性、可对比性和量化评测，而非具体实现或应用场景。

为什么要做这个 Benchmark

- 第一，中期阶段原有纵向结果说服力不足，不适合直接得出“方法效果显著提升”的结论。

- 第二，本项目当前最成熟的成果其实是系统工程闭环，而不是最终生物效果闭环。
- 第三，答辩需要一组更严谨的证据，来说明系统现在已经具备可控执行、人工确认、失败恢复和证据沉淀能力。

本项目中的 Benchmark

结合本项目来说，这套 benchmark 的对象不是最终蛋白质设计质量，而是系统的关键机制是否成立。它验证的是：

- 门控与等待态是否能正常触发
- 人工确认是否能进入正式决策链
- 失败后是否能按照 `retry` → `patch` → `replan` 继续恢复
- 路由、执行、汇总是否能闭环
- 日志、快照、报告是否能够完整落盘

补充说明：答辩追问回答

问题一：系统只接入了一个 LLM，为什么说多 Agent，而不是基于 Agent 的工作流？Agent 是什么？

1. 可以先给出的简短回答 从模型接入上看，目前系统主要接入的是一个 LLM provider；但从系统结构上看，它仍然是多 Agent，因为任务被拆成了多个具有独立职责、独立输入输出契约和独立权限边界的 Agent。

换句话说，多 Agent 并不要求每个 Agent 都绑定一个不同的大模型接口，更关键的是是否存在稳定的角色分工、边界约束和协作闭环。

2. 为什么本项目可以称为多 Agent 本项目中至少有四类明确角色：

- PlannerAgent
 - 负责输出 Plan、PlanPatch 或 Replan
 - 不能执行工具
- ExecutorAgent
 - 负责真正执行步骤、处理 `retry/patch/replan` 触发
 - 不能代替人工做确认决策
- SafetyAgent

- 负责给出 allow / warn / block
- 不能执行工具，也不能修改计划
- SummarizerAgent
 - 负责结果汇总与报告生成
 - 不能重跑工具，也不能改变状态决策

这个边界在设计契约里有明确要求，也在代码里对应为不同模块和不同调用职责，而不是一个单体 Agent 在脚本里串完所有事情。

3. 它与“基于 Agent 的工作流”是什么关系 更准确地说，本项目可以表述为：

一个由显式 FSM 控制的多 Agent 工作流系统。

也就是说，它既有工作流特征，也有多 Agent 特征：

- 说它是 工作流系统
 - 是因为整体执行遵循显式状态机和固定生命周期
- 说它是 多 Agent 系统
 - 是因为规划、执行、安全审查、总结这些能力不是由一个单体角色完成，而是由多个 Agent 协作完成

它不是多个完全独立自由对话的 LLM 代理，而是一个在显式状态机控制下、具有多 Agent 角色分工的工作流系统。

5. 答辩时可直接说的话 我这里说多 Agent，强调的是角色边界和协作闭环，而不是每个 Agent 都接一个不同的大模型。当前系统虽然主要接入一个 LLM provider，但 Planner、Executor、Safety、Summarizer 四类 Agent 的职责是分开的，输入输出契约和权限边界也是分开的，因此它不是单体 Agent 加脚本串联，而是一个由显式状态机控制的多 Agent 工作流系统。

问题二：如何保证 retry 和 replan 的结果可追溯，并且能够与失败执行对应起来？结合代码来说

1. 可以先给出的简短回答 核心做法是把失败步骤、恢复候选、状态转移、人工决策和最终重规划都写成结构化数据，并同时落到 StepResult、EventLog 和 TaskSnapshot 三层证据里。这样 replan 不是一个孤立动作，而是可以沿着失败步骤一路追溯回去。

2. 追溯链条是怎么建立的 本项目不是只记录“最后是否重规划”，而是把恢复链条拆成几个可对账的对象：

- 失败步骤本身
 - 在 `StepResult` 中保留 `step_id`、`tool`、`failure_type`、`error_message`
- patch 与 recovery 元数据
 - 在 `metrics["patch"]` 和 `metrics["recovery"]` 中记录
 - 包括 `from_tool`、`to_tool`、`recovery_layer`、`candidate_id`、`upgrade_reason`
- 事件日志
 - 记录 `STEP_FAILED`、`REPLACE_TOOL`、`RECOVERY_ESCALATED`、`DECISION_APPLIED`、`WAITING_EXIT`
- 快照
 - 在进入 `WAITING_*` 前写入 `TaskSnapshot`
 - 保留当时的计划、进度和 `PendingAction`

这样一来，就能从“最终进入 `replan`”反向追到：

- 是哪一个 `step` 失败
- 为什么失败
- 中间尝试过哪一层 `patch`
- 是否发生过工具替换
- 是因为 `patch_failed` 还是 `patch_high_risk` 升级到 `replan`
- 人工是否做过确认以及选了哪个候选

3. 代码上是如何实现的 第一层：失败结果本身带 `trace`。

- 在 `_attach_patch_meta` 中，会把 `patch` 信息写入 `patched_result.metrics["patch"]`。
- 同时也会把 `recovery` 信息写入 `patched_result.metrics["recovery"]`，包括：
 - `recovery_layer`
 - `capability_id`
 - `from_tool`
 - `to_tool`
 - `candidate_id`
 - `upgrade_reason`

第二层：`patch` 升级到 `replan` 时会发出专门事件。

- 在 `_emit_recovery_escalation_event` 中，会写入 `RECOVERY_ESCALATED` 事件。

- 事件数据里包含：
 - reason
 - detail
 - recovery

第三层：失败步骤事件会把 patch/recovery trace 带出去。

- 在 plan_runner.py#L746 的 _emit_step_event 中，失败步骤会写成 STEP_FAILED。
- 在 plan_runner.py#L781 的 _build_step_trace_data 中，会把 patch、recovery、failure_code 等字段统一放进事件 data。

第四层：进入等待态前先写 PendingAction 和快照。

- 在 pending_action.py#L79 的 enter_waiting_state 中，系统会先写入 PendingAction。
- 同时写入 WAITING_ENTER 事件和 TaskSnapshot，对应 pending_action.py#L147 和 pending_action.py#L181。
- 这保证了系统即便中断，也能恢复到当时的等待决策上下文。

第五层：Decision 与等待态退出也有审计链。

- 在 decision_apply.py 开始，系统会记录 DECISION_SUBMITTED、DECISION_APPLIED、WAITING_EXIT。
- 对应的结构化 EventLog 构造在 event_log_factory.py

4. 为什么这样就能对应到失败执行 因为这条链里有几个关键字段是贯通的：

- task_id
- step_id
- pending_action_id
- decision_id
- candidate_id
- from_tool / to_tool
- upgrade_reason

这些字段会在失败结果、恢复事件、等待态事件和决策事件里反复出现，所以最终看到一次 re-plan 时，可以把它和之前的失败步骤以及中间 patch 尝试对应起来，而不是只知道“系统后来又重新规划了一次”。

5. 测试上如何证明它成立 这一点不是只靠口头描述，集成测试也有覆盖。

- 在 `test_recovery_layered_patch.py` 开始的测试里, 会验证 `patch` 失败后升级到 `replan`, 并检查:
 - `context.status == WAITING_REPLAN`
 - `record.status == WAITING_REPLAN_CONFIRM`
 - `recovery["upgrade_reason"] == "patch_failed"`
 - 时间线里确实存在 `RECOVERY_ESCALATED`
- 在同一文件 `test_recovery_layered_patch.py` 开始的测试里, 也验证了高风险 `patch` 升级到 `replan` 的链路。

6. 答辩时可直接说的话 为了保证 `retry` 和 `replan` 的结果可追溯, 我没有把恢复逻辑做成黑盒跳转, 而是把失败步骤、`patch` 元数据、`replan` 升级原因、等待态、人工决策和快照都做成结构化记录。代码里一方面在 `StepResult` 的 `metrics` 中保留 `patch` 和 `recovery` 字段, 另一方面在 `EventLog` 中记录 `STEP_FAILED`、`RECOVERY_ESCALATED`、`DECISION_APPLIED` 等事件, 并在进入 `WAITING` 状态前写入 `TaskSnapshot`。因此最终一次 `replan` 可以明确追溯到是哪一个 `step` 失败、尝试过哪些 `patch`、为什么升级, 以及对应的人机决策链。

补充说明四: Benchmark 实验是如何做的

1. 可以先给出的简短回答

这套 `benchmark` 不是传统意义上的公开数据集跑分, 而是我把项目里已经存在的可复现场景整理成了一套机制验证基准。它验证的不是最终蛋白质设计效果, 而是系统在门控、人工确认、失败恢复、路由、汇总和证据沉淀这些关键机制上是否已经稳定成立。

2. 这套 `benchmark` 的基本思路

中期阶段原有纵向结果的说服力有限, 因此不适合直接得出“方法效果显著提升”的结论。针对这个问题, 我没有再去包装旧结果, 而是换了一个更符合当前项目成熟度的问题:

当前这个系统最核心的工程能力, 到底有没有被稳定验证?

所以这套 `benchmark` 的目标不是比较谁的最终蛋白质效果最好, 而是检验系统机制闭环是否已经成立。

3. benchmark 验证的对象是什么

这组实验验证的对象主要不是单条蛋白质序列，而是系统中的关键运行对象和证据链，包括：

- Plan / PlanPatch / Replan
- PendingAction
- Decision
- StepResult
- EventLog
- TaskSnapshot
- DesignResult

也就是说，它验证的是这些对象之间的关系是否能在真实运行中成立，例如：

- 是否真的进入 WAITING_*
- 是否真的触发人工确认
- patch 失败后是否真的升级到 replan
- 关键日志、快照和报告是否真的落盘

4. 这套 benchmark 是怎么组织出来的

我先把系统能力拆成了 5 个实验家族：

- Gate
- HITL & Audit
- Recovery
- Execution & Summary
- Planner Routing

然后在配置文件里，把仓库中已有的集成测试逐个登记成场景清单。这个配置里，每个场景都明确写出：

- id
- family
- runner
- target
- capabilities
- 需要检查的 artifacts

- 需要检查的 `expected_signals`

所以这套 `benchmark` 不是临时手工截图汇总，而是正式定义过的场景集合。

5. 具体用了哪些案例

一共纳入了 17 个可复现场景，主要来自仓库中已经存在的集成测试。

例如：

Gate 家族：

- `plan_gate_paths`
- `patch_gate_paths`

对应测试来自：

- `test_candidate_score_gate.py`

HITL & Audit 家族：

- `waiting_enter_event`
- `decision_apply_events`
- `fsm_reconstruction`
- `six_stage_hitl_replay`

对应测试来自：

- `test_event_log_integration.py`
- `test_s6_control_layer_e2e.py`

Recovery 家族：

- `s3_failure_to_replan`
- `layered_patch_tool_success`
- `layered_patch_remote_local`
- `patch_failure_to_replan`
- `high_risk_patch_to_replan`

对应测试主要来自：

- `test_recovery_layered_patch.py`

- `test_s6_control_layer_e2e.py`

Execution & Summary 家族:

- `mock_remote_full_flow`
- `esmfold_summarizer_integration`
- `summarizer_empty_results`

Planner Routing 家族:

- `dual_route_failures`
- `dual_route_recovery`
- `dual_route_exec_rate`

这说明 `benchmark` 不是只看一个 `demo`，而是把多个已有能力验证点统一组织起来。

6. 是通过什么脚本运行的

运行脚本是:

- `run_midterm_mechanism_benchmark.py`

我实际执行的命令是:

```
uv run python scripts/run_midterm_mechanism_benchmark.py --run-id midterm-mechanism-benchmark
```

脚本会做几件事:

1. 读取 `benchmark` 配置。
2. 逐个场景构造命令。
3. 当前所有场景统一通过 `uv run pytest <target> -q` 执行。
4. 每个场景执行前，先清理它声明的旧产物，避免历史文件干扰统计。
5. 执行完成后，统计：
 - 是否通过
 - 运行时长
 - 声明的 `artifact` 是否存在
 - 声明的 `signal` 是否满足
6. 最后统一写出 CSV、JSON、Markdown 报告和 SVG 图表。

7. 什么是 artifacts 和 signals

这是这套 benchmark 比普通“pytest 通过”更严一点的地方。

artifacts 指的是这个场景理论上应该生成的证据文件，例如：

- event_log
- snapshot
- report

比如在 six_stage_hitl_replay 场景里，配置里明确要求检查：

- data/logs/int_s6_patch_decision_replay_done.jsonl
- data/snapshots/int_s6_patch_decision_replay_done.jsonl

也就是说，不仅测试要通过，而且证据文件还必须真实存在。

signals 指的是从事件时间线里进一步核对的关键机制信号。对应逻辑在：

- midterm_mechanism_benchmark.py

目前检查的 signal 包括：

- waiting_chain_complete
- done_transition
- patching_transition
- replace_tool_event
- recovery_escalated_patch_failed
- recovery_escalated_patch_high_risk
- s6_replan_step_failed

这意味着某个场景不能只靠“测试命令返回 0”就算完成，还要满足证据和关键信号。

8. 最后生成了什么结果

脚本会在输出目录里生成完整结果包：

- midterm-mechanism-benchmark-r2

其中包括：

- scenario_results.csv

- family_summary.csv
- capability_summary.csv
- artifact_summary.csv
- signal_results.csv
- midterm_mechanism_benchmark_report.md

以及三张图表：

- family_summary.svg
- capability_coverage.svg
- artifact_support.svg

你这次 PPT 实际用了其中两张：

- 第 10 页：family_summary.svg
- 第 11 页：artifact_support.svg

9. 为什么这套实验适合中期答辩

因为这套实验和中期阶段的项目成熟度是匹配的。

中期阶段，最成熟的成果不是“最终蛋白质设计效果已经完全验证”，而是：

- 系统已经能控
- 已经能停
- 已经能让人参与
- 已经能从失败中恢复
- 已经能留下可追溯证据

所以这套 benchmark 的作用，是把这些最成熟、最稳定、最容易复现的工程能力，正式整理成一套实验结果，而不是继续依赖说服力不足的旧纵向结果。

10. 答辩时可直接说的话

这套 benchmark 不是传统意义上的公开数据集跑分，而是我把仓库里已经存在的可复现场景整理成了一套机制验证基准。它一共包含 17 个场景，覆盖 Gate、HITL and Audit、Recovery、Execution and Summary、Planner Routing 这 5 个实验家族。运行时统一通过脚本逐个执行 pytest 场景，并额外检查日志、快照、报告等证据文件是否落

盘，以及 waiting chain、patching transition、recovery escalated 等关键信号是否成立。所以它验证的不是最终蛋白质设计效果，而是当前系统最核心的控制闭环、恢复闭环和审计闭环是否已经稳定成立。